

```
import pandas as pd

emotion_df = pd.read_csv("/content/emotion-emotion_69k[1].csv")
conv_df = pd.read_csv("mental_health_conversations.csv")

print("Emotion Dataset Columns:", emotion_df.columns)
print(emotion_df.head())

print("\nConversation Dataset Columns:", conv_df.columns)
print(conv_df.head())
```

Emotion Dataset Columns: Index(['Unnamed: 0', 'Situation', 'emotion', 'empathetic_dialogues', 'labels', 'Unnamed: 5', 'Unnamed: 6'], dtype='object')

Unnamed: 0	Situation	emotion
0	I remember going to the fireworks with my best...	sentimental
1	I remember going to the fireworks with my best...	sentimental
2	I remember going to the fireworks with my best...	sentimental
3	I remember going to the fireworks with my best...	sentimental
4	I remember going to the fireworks with my best...	sentimental

empathetic_dialogues \

```
0 Customer :I remember going to see the firework...
1 Customer :This was a best friend. I miss her.\...
2 Customer :We no longer talk.\nAgent :
3 Customer :Was this a friend you were in love w...
4 Customer :Where has she gone?\nAgent :
```

labels	Unnamed: 5	Unnamed: 6
0	Was this a friend you were in love with, or ju...	NaN
1	Where has she gone?	NaN
2	Oh was this something that happened because of...	NaN
3	This was a best friend. I miss her.	NaN
4	We no longer talk.	NaN

Conversation Dataset Columns: Index(['question', 'answer', 'source', 'source_dataset', 'statement', 'status'], dtype='object')

question \

```
0 I guess I tried to be present and supported, b...
1 I think partly due to the pressure I felt from...
2 Yes, I definitely allowed myself to feel the p...
3 No, I think this conversation is really helpfu...
4 I think you are right. It is difficult to shak...
```

answer \

```
0 Remember, small efforts can make a big differe...
1 I heard it. The expectations of us in childhoo...
2 It is great to give yourself the space to pond...
3 Nothing. It is honored to be supported on the ...
4 I heard your level of anxiety. Let's rearrange...
```

source	source_dataset
0 combined_mental_health_processed.csv	Student-Mental-Health-Counseling-20K
1 combined_mental_health_processed.csv	Student-Mental-Health-Counseling-20K
2 combined_mental_health_processed.csv	Student-Mental-Health-Counseling-20K
3 combined_mental_health_processed.csv	Student-Mental-Health-Counseling-20K
4 combined_mental_health_processed.csv	Student-Mental-Health-Counseling-20K

statement	status
0	NaN Normal
1	NaN Normal
2	NaN Normal
3	NaN Normal
4	NaN Normal

```
import re
import nltk
nltk.download('stopwords')

from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = str(text).lower()
    text = re.sub(r"http\S+", "", text)
    text = re.sub(r"[^a-zA-Z]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()

    words = text.split()
    words = [w for w in words if w not in stop_words]

    return " ".join(words)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

```
# CHANGE column names if different
TEXT_COL = "Situation"
LABEL_COL = "emotion"

#emotion_df['clean_text'] = emotion_df[TEXT_COL].apply(preprocess)

print(emotion_df[[TEXT_COL, LABEL_COL]].head())
```

```

              Situation      emotion
0  I remember going to the fireworks with my best... sentimental
1  I remember going to the fireworks with my best... sentimental
2  I remember going to the fireworks with my best... sentimental
3  I remember going to the fireworks with my best... sentimental
4  I remember going to the fireworks with my best... sentimental

```

```
print(emotion_df[[TEXT_COL, LABEL_COL]].head(500))
```

```

              Situation      emotion
0  I remember going to the fireworks with my best... sentimental
1  I remember going to the fireworks with my best... sentimental
2  I remember going to the fireworks with my best... sentimental
3  I remember going to the fireworks with my best... sentimental
4  I remember going to the fireworks with my best... sentimental
..      ...
495 I was thrown a surprise party for my 30th birt... surprised
496 I was thrown a surprise party for my 30th birt... surprised
497 I was thrown a surprise party for my 30th birt... surprised
498 I was told my cat only has a few weeks to live.      sad
499 I was told my cat only has a few weeks to live.      sad

```

```
[500 rows x 2 columns]
```

```
unique_emotions = emotion_df[LABEL_COL]
print(unique_emotions)
```

```

0      Anxiety
1      Anxiety
2      Anxiety
3      Anxiety
4      Anxiety
...
53038 Anxiety
53039 Anxiety
53040 Anxiety
53041 Anxiety
53042 Anxiety
Name: status, Length: 53043, dtype: object

```

```
emotion_df = emotion_df.drop_duplicates()
```

Start coding or generate with AI.

```
emotion_df[TEXT_COL]
```

```

              statement
0      oh my gosh
1  trouble sleeping, confused mind, restless hear...
2  All wrong, back off dear, forward doubt. Stay ...
3  I've shifted my focus to something else but I'...
4  I'm restless and restless, it's been a month n...
...
53038 Nobody takes me seriously I've (24M) dealt wit...
53039 selfishness "I don't feel very good, it's lik...
53040 Is there any way to sleep better? I can't slee...
53041 Public speaking tips? Hi, all. I have to give ...
53042 I have really bad door anxiety! It's not about...
53043 rows x 1 columns

dtype: object

```

Start coding or generate with AI.

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# Drop rows with NaN values in the LABEL_COL and TEXT_COL before splitting and training
emotion_df = emotion_df.dropna(subset=[LABEL_COL, TEXT_COL])

# Split the data into training and testing sets

vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1,2))

X = vectorizer.fit_transform(emotion_df[TEXT_COL])
y = emotion_df[LABEL_COL]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

```

▼ **LogisticRegression** ⓘ ?
LogisticRegression(max_iter=200)

```

from sklearn.metrics import accuracy_score

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

```

Accuracy: 0.7184188133364277

Start coding or generate with AI.

```
print(conv_df.columns)
```

```

Index(['question', 'answer', 'source', 'source_dataset', 'statement',
      'status'],
      dtype='object')

```

```

conv_df = conv_df[['question', 'answer']]
print(conv_df.head())

```

```

      question \
0  I guess I tried to be present and supported, b...
1  I think partly due to the pressure I felt from...
2  Yes, I definitely allowed myself to feel the p...
3  No, I think this conversation is really helpfu...
4  I think you are right. It is difficult to shak...

      answer
0  Remember, small efforts can make a big differe...
1  I heard it. The expectations of us in childhoo...
2  It is great to give yourself the space to pond...
3  Nothing. It is honored to be supported on the ...
4  I heard your level of anxiety. Let's rearrange...

```

```
conv_df['clean_question'] = conv_df['question'].astype(str).apply(preprocess)
```

```

/tmp/ipykernel_12593/4046596489.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

```

```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
conv_df['clean_question'] = conv_df['question'].astype(str).apply(preprocess)

```

```

from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1,2))

X = vectorizer.fit_transform(conv_df['clean_question'])

```

```

from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

def get_therapist_response(user_input):
    user_clean = preprocess(user_input)

    user_vec = vectorizer.transform([user_clean])

    similarity = cosine_similarity(user_vec, X)

    index = np.argmax(similarity)

    return conv_df['answer'].iloc[index]

```

```

def chatbot(user_input):

    clean = preprocess(user_input)

    # emotion detection (your model)
    emotion = model.predict(vectorizer.transform([clean]))[0]

```

```
# response generation
response = get_therapist_response(user_input)

return emotion, response
```

```
while True:
    user = input("you :")

    if user.lower() == "exit":
        break

    emotion, response = chatbot(user)

    print("Detected Emotion:", emotion)
    print("Therapist:", response)
```

```
import pickle

# Save the trained Logistic Regression model
with open('emotion_model2.pkl', 'wb') as f:
    pickle.dump(model, f)

# Save the TfidfVectorizer
with open('tfidf_vectorizer2.pkl', 'wb') as f:
    pickle.dump(vectorizer, f)

# Save the conversation DataFrame
conv_df.to_csv('conversation_data.csv', index=False)

print("Model, vectorizer, and conversation data saved successfully!")

Model, vectorizer, and conversation data saved successfully!
```

Now that the necessary components are saved, you can create a Python file (e.g., `streamlit_app.py`) in VS Code with the following structure. Make sure the `(emotion_model.pkl)`, `(tfidf_vectorizer.pkl)` and `(conversation_data.csv)` files are in the same directory as your Streamlit app.

```
# Install Streamlit if not already installed (for Colab environment)
try:
    import streamlit
except ImportError:
    !pip install streamlit --quiet

import streamlit as st
import pandas as pd
import re
import nltk
import pickle
from nltk.corpus import stopwords
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Ensure stopwords are downloaded (run once)
try:
    stopwords.words('english')
except LookupError:
    nltk.download('stopwords')

stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = str(text).lower()
    text = re.sub(r"http\S+", "", text)
    text = re.sub(r"[^a-zA-Z]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    words = text.split()
    words = [w for w in words if w not in stop_words]
    return " ".join(words)

@st.cache_resource
def load_artifacts():
    # Load the trained model
    with open('emotion_model.pkl', 'rb') as f:
        loaded_model = pickle.load(f)

    # Load the TfidfVectorizer
    with open('tfidf_vectorizer.pkl', 'rb') as f:
        loaded_vectorizer = pickle.load(f)

    # Load the conversation DataFrame
    loaded_conv_df = pd.read_csv('conversation_data.csv')

    # Ensure 'clean_question' column exists, preprocess if not present
    if 'clean_question' not in loaded_conv_df.columns:
        # Assuming 'question' column exists in conversation_data.csv
        loaded_conv_df['clean_question'] = loaded_conv_df['question'].astype(str).apply(preprocess)

    loaded_X_conv = loaded_vectorizer.transform(loaded_conv_df['clean_question'])

    return loaded_model, loaded_vectorizer, loaded_conv_df, loaded_X_conv
```

```

model, vectorizer, conv_df, X_conv = load_artifacts()

def get_therapist_response(user_input, loaded_conv_df, loaded_vectorizer, loaded_X_conv):
    user_clean = preprocess(user_input)
    user_vec = loaded_vectorizer.transform([user_clean])
    similarity = cosine_similarity(user_vec, loaded_X_conv)
    index = np.argmax(similarity)
    return loaded_conv_df['answer'].iloc[index]

def chatbot(user_input, loaded_model, loaded_vectorizer, loaded_conv_df, loaded_X_conv):
    clean_input = preprocess(user_input)
    emotion = loaded_model.predict(loaded_vectorizer.transform([clean_input]))[0]
    response = get_therapist_response(user_input, loaded_conv_df, loaded_vectorizer, loaded_X_conv)
    return emotion, response

st.title("Mental Health Chatbot")
st.write("Hello! I'm here to listen. How are you feeling today?")

user_input = st.text_input("You:", key="user_input")

if user_input:
    emotion, response = chatbot(user_input, model, vectorizer, conv_df, X_conv)
    st.write(f"***Detected Emotion:** {emotion}")
    st.write(f"***Therapist:** {response}")

st.write("Type 'exit' to end the conversation (though you can just close the tab too).")

```

```

----- 9.2/9.2 MB 63.2 MB/s eta 0:00:00
----- 11.3/11.3 MB 105.8 MB/s eta 0:00:00
2026-05-02 08:16:52.644 WARNING streamlit.runtime.scriptrunner_utils.script_run_context: Thread 'MainThread': missing ScriptRunContext! This warning can be ignored
-----

```

```

KeyError                                Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3804         try:
-> 3805             return self._engine.get_loc(casted_key)
   3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'clean_question'

```

The above exception was the direct cause of the following exception:

```

KeyError                                Traceback (most recent call last)
----- 6 frames -----
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3810         ):
   3811             raise InvalidIndexError(key)
-> 3812         raise KeyError(key) from err
   3813     except TypeError:
   3814         # If we have a listlike key, _check_indexing_error will raise

KeyError: 'clean_question'

```

To run this Streamlit app, save the code above as `streamlit_app.py` in the same directory as your `.pkl` and `.csv` files. Then, open your terminal in that directory and run:

```
streamlit run streamlit_app.py
```

Start coding or generate with AI.

Start coding or generate with AI.